

Strings

Text, In Depth

Session 7

What We're Covering Today

- Strings as sequences — indexing, slicing, immutability, looping
- Core methods: case, whitespace, split() & join()
- Searching: find(), startswith(), endswith(), count()
- Checking content: isdigit(), isalpha(), isalnum()
- f-strings in depth, and number formatting
- Concatenation vs. f-strings vs. join() — which wins
- Real tasks: validation, parsing, and a data cleaner



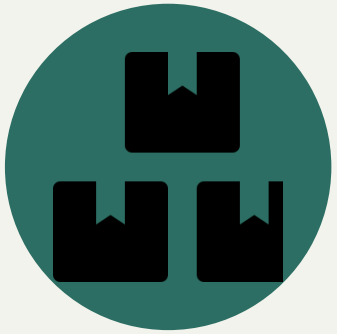
PART 1

Strings Are Sequences

Everything you learned about lists applies here too.



Strings Are Sequences of Characters

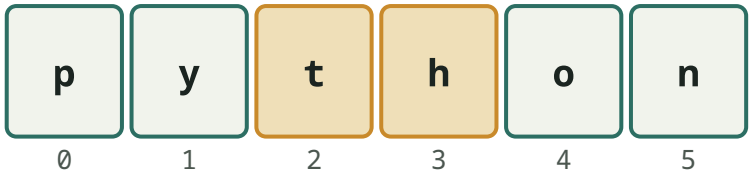


Ordered.
Indexable.
Immutable.

- A string is an ordered sequence of characters — very similar to a list of single letters
- This is exactly what "Where Do Strings Fit?" back in Session 5 was hinting at
- Because it's a sequence, indexing and slicing both work exactly the way they do on a list
- The one real difference: a string can never be changed after it's created — more on that shortly

Indexing & Slicing a String

```
word = "python"
```



- word[0] → "p" word[-1] → "n"
- word[2:4] → "th" — same slicing rules as a list, start included, stop excluded
- Every trick from Lists — negative indices, steps, [::-1] to reverse — works here too



Strings Are Immutable

- Just like a tuple, a string can never be changed in place once it's created
- Trying to assign to a specific character raises a TypeError
- To "change" a string, you always build a brand new one instead

```
word = "python"

word[0] = "P"
# TypeError: 'str' object does not
# support item assignment

# the correct way — build a new string
word = "P" + word[1:]
print(word)   # "Python"
```

Looping Over a String

- A for loop walks through a string one character at a time, exactly like it does with a list
- This is the foundation for almost every text-processing task you'll ever write

```
word = "python"

for letter in word:
    print(letter)

# p y t h o n  (each on its own line)
```

Combining Strings: + and *

- + joins two strings together into one new string — this is called concatenation
- * repeats a string a given number of times

```
first = "Py"
second = "thon"
print(first + second)   # "Python"

print("ab" * 3)        # "abababab"
```

Practice Set — Strings as Sequences

Try these yourself first — no code shown, that's the point

1. Take a word and print its first and last character using indexing
2. Reverse a string using slicing ([::-1])
3. Loop through a string and count how many vowels it contains
4. Check whether a word is a palindrome by comparing it to its own reverse
5. Try changing one character in a string directly, and read the error Python gives you



PART 2

Core String Methods

Changing Case

- .upper() and .lower() convert every character, without changing the original string
- .title() capitalizes the first letter of every word — handy for names
- None of these change the original — they return a brand new string, since strings are immutable

```
name = "ahmed ali"

print(name.upper())   # "AHMED ALI"
print(name.lower())   # "ahmed ali"
print(name.title())   # "Ahmed Ali"

print(name)           # "ahmed ali"  <- unchanged
```



Removing Whitespace: strip()

- .strip() removes whitespace (spaces, tabs, newlines) from both ends of a string
- .lstrip() and .rstrip() do the same, but only from the left or right side
- Extremely common when cleaning real user input or messy data files

```
raw = "   ahmed ali   "

print(raw.strip())    # "ahmed ali"
print(raw.lstrip())   # "ahmed ali   "
print(raw.rstrip())   #"   ahmed ali"
```



.split() — Breaking a String Apart

- split() breaks a string into a list of pieces, using a separator you choose
- With no argument, it splits on any whitespace by default
- This is exactly how you'd start parsing a line of comma-separated data

```
sentence = "the quick brown fox"
words = sentence.split()
print(words)   # ['the', 'quick', 'brown', 'fox']

row = "Ahmed,25,Karachi"
fields = row.split(",")
print(fields)   # ['Ahmed', '25', 'Karachi']
```

.join() — Gluing a List Back Together

- join() is the reverse of split() — it combines a list of strings into one string
- You call it ON the separator you want, and pass it the list
- A very common beginner mix-up: it's "separator".join(list), not list.join("separator")

```
words = ["the", "quick", "brown", "fox"]

sentence = " ".join(words)
print(sentence)   # "the quick brown fox"

csv_line = ",".join(["Ahmed", "25", "Karachi"])
print(csv_line)   # "Ahmed,25,Karachi"
```



.replace() — Swapping Text

- replace() finds every occurrence of one piece of text and swaps it for another
- Returns a new string — remember, the original never changes

```
sentence = "I like cats. Cats are great."

print(sentence.replace("Cats", "Dogs"))
# "I like cats. Dogs are great."
# note: only exact-case matches get replaced
```

A Practice Set — Core Methods

Try these yourself first — no code shown, that's the point

1. Take a messy string with extra spaces on both ends, and clean it using strip()
2. Convert a full sentence to uppercase and then to title case
3. Split a sentence into a list of words
4. Join a list of words back into one sentence with spaces between them
5. Replace one specific word in a sentence with a different word



PART 3

Searching Inside Strings

.find() vs. .index()

- Both search for a piece of text and return the position where it starts
- The difference is what happens when nothing is found
- find() returns -1 quietly; index() raises a ValueError and stops your program

```
sentence = "the quick brown fox"

print(sentence.find("brown"))    # 10
print(sentence.find("zebra"))    # -1

print(sentence.index("zebra"))
# ValueError: substring not found
```

.startswith() and .endswith()

- Check whether a string begins or ends with a specific piece of text
- Extremely common for checking file names, commands, and URLs

```
filename = "report_final.csv"

print(filename.endswith(".csv"))     # True
print(filename.startswith("report")) # True
print(filename.endswith(".xlsx"))    # False
```

.count() and in / not in

- count() tells you how many times a piece of text appears — same idea as list/tuple count()
- in and not in check for existence anywhere in the string, without caring how many times

```
sentence = "the quick brown fox jumps over the lazy dog"

print(sentence.count("the"))      # 2
print("fox" in sentence)          # True
print("cat" not in sentence)      # True
```

Practice Set — Searching

Try these yourself first — no code shown, that's the point

1. Check whether a filename ends with ".py" using endswith()
2. Find the position of a specific word inside a sentence using find()
3. Count how many times a specific letter appears in a word
4. Check whether a keyword exists somewhere inside a paragraph using in



PART 4

Checking What Kind of Text You Have



Content-Checking Methods



.isdigit()

True only if every character is a digit



.isalpha()

True only if every character is a letter



.isalnum()

True if every character is a letter or digit



.isspace()

True only if the string is entirely whitespace



Real-World Use: Input Validation

- These content checks are what make real input validation possible
- Combined with the while loop pattern from last session, you can force genuinely valid input

```
age_text = input("Enter your age: ")

while not age_text.isdigit():
    print("Please enter numbers only.")
    age_text = input("Enter your age: ")

age = int(age_text)
print("Thanks! You are", age, "years old.")
```

Practice Set — Content Checks

Try these yourself first — no code shown, that's the point

1. Check whether a given string is entirely numeric using isdigit()

2. Check whether a username contains only letters and numbers using isalnum()

3. Write a check that flags whether a piece of input was left completely blank



PART 5

Formatting Strings

f-strings — A Recap, and a Step Further

- f-strings let you drop a variable — or any expression — directly inside { }
- You can even do math or call a method right inside the braces

```
name = "Ahmed"
score = 88

print(f"{name} scored {score}")
# "Ahmed scored 88"

print(f"{name.upper()} scored {score + 2} after review")
# "AHMED scored 90 after review"
```

Number Formatting Inside f-strings

- A colon inside { } lets you control exactly how a number is displayed
- :.2f rounds a decimal number to exactly 2 places — essential for money and measurements
- , adds thousands separators, useful for large numbers

```
price = 19.6789
print(f"Price: ${price:.2f}")
# "Price: $19.68"

population = 1500000
print(f"Population: {population:,}")
# "Population: 1,500,000"
```

Concatenation vs. f-strings vs. join()

Use f-strings when...

- You're building a sentence mixing text and variables
- They're almost always the most readable choice
- Example: f"{name} is {age} years old"

Use join() when...

- You already have a list of strings to combine
- You need a specific separator between many pieces
- Example: ",".join(fields) for a CSV row

Escape Characters — A Proper Recap

- First seen back in the operators session — here's where they really belong
- \n and \t show up constantly once you're formatting real text output

```
report = f"Name:\t{'Ahmed'}\nScore:\t{88}"
print(report)

# Name:    Ahmed
# Score:   88
```

Practice Set — Formatting

Try these yourself first — no code shown, that's the point

1. Use an f-string to print your name and age in one sentence

2. Format a decimal number to exactly 2 decimal places inside an f-string

3. Build the same sentence three ways — concatenation, f-string, and join() — and compare



REAL TASKS

These Are Things You'll Actually Build

Not toy examples — validation, parsing, and cleaning are core, everyday data work.

Today's Tasks — Part A: Validation

Small tasks — do them in order

1. Write a username validator: valid only if it contains letters and numbers only, no spaces or symbols (use isalnum())

2. Write a password strength checker: valid only if it is at least 8 characters long AND contains at least one digit

3. Write a program that takes a messy full name (extra spaces, random casing) like " aHMED   ali " and outputs it cleanly as "Ahmed Ali"

Today's Tasks — Part B: Parsing & Formatting

Small tasks — do them in order

- **4** Given a line like "Ahmed,25,Karachi", split it and print each field with a label (Name / Age / City)
- **5** Write a basic email checker: valid only if the text contains "@" and ends with ".com"
- **6** Given a list of product names and prices, loop through them and print a clean receipt with every price formatted to 2 decimal places



PART C

Put It All Together

Two real projects — the exact kind of text-cleaning work you'd actually reach for AI to help with in EDA.

Mini Project 1: Data Cleaner



This is real EDA work

- You're given a messy, user-submitted string like " JOHN   smith!! "
- Clean it completely: remove extra whitespace, fix the casing to Title Case, and strip out trailing punctuation
- Chain multiple string methods together to do this in as few lines as possible
- Print both the original and the cleaned version, side by side, so the transformation is obvious

Mini Project 2: Simple Log Parser



A taste of real logging data

- You're given a single fake log line: "ERROR: Connection failed at 14:32"
- Use split() to separate it into the log level ("ERROR"), the message, and the timestamp
- Print each piece separately, clearly labeled
- Bonus: use startswith() to check whether the log level is "ERROR" and print a warning if so



RECAP

Strings behave like lists — but they never let you change them.
Every method today builds toward reading and cleaning real data.

Next session: dictionaries — the last major data structure, and a very natural fit now that loops exist.